

REINFORCEMENT LEARNING: ЗАДАЧА ЗАПРАВКИ ТС НА МАРШРУТЕ СЛЕДОВАНИЯ

Ложкин Алексей,
специалист по исследованию операций



ОБО МНЕ

LA Optimization

11 лет

Занимаюсь разработкой
оптимизационных решений

LA Optimization

Частный консультант по
математическому
моделированию и оптимизации
бизнеса

MOS

Make Optimization Simple - блог-
проект про математическую
оптимизацию

Области моделирования:

- Логистика
- Цепочки поставок
- Ресурсное планирование
- Производственное планирование
- Маршрутизация

Разрабатывал
оптимизационные
решения для



Ложкин
Алексей

ЗАДАЧА

ОСП А



A3C 3

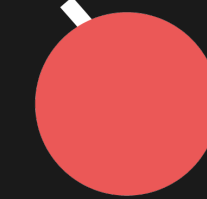
Ограничения

- Минимальный остаток в баке
- Максимальная вместимость бака
- Минимальная заправка на АЗС
- Остаток топлива в конце маршрута
- Объем заправки 0%, 25%, 50%, 75%, 100% от свободного места в баке

A3C 5

A3C 4

ОСП Б



МОДЕЛЬ НЕЛИНЕЙНОГО ПРОГРАММИРОВАНИЯ

Индексы

$i \in I$ – индексы АЗС

Постоянные

c_i - объем потребления топлива с $i - 1$ до i

p_i - стоимость топлива на АЗС i

v_m - минимальный объем заправки на АЗС

s_0, s_n - объем топлива в начале и конце маршрута

s_{lb}, s_{ub} - минимальный и максимальный объем топлива

Переменные

s_i - вещественная переменная, остаток топлива в баке по заправки на АЗС i

v_i - объем заправки на АЗС i

q_i - кол-во квартилей заправки на АЗС i

b_i - индикатор наличия заправки на АЗС i

1. Минимизация стоимости заправок на маршруте

$$\min \sum_{i \in I} p_i v_i$$

2. Баланс остатков топлива в баке

$$s_i = s_{i-1} - c_i + v_i, \quad \forall i \in I$$

3. Ограничение минимального остатка в баке

$$s_{i-1} - c_i \geq s_{lb}, \quad \forall i \in I$$

4. Остаток в конце маршрута

$$s_{|I|} - c_{|I|+1} \geq s_n$$

5. Ограничение минимальной заправки

$$v_i \geq v_m b_i, \quad \forall i \in I$$

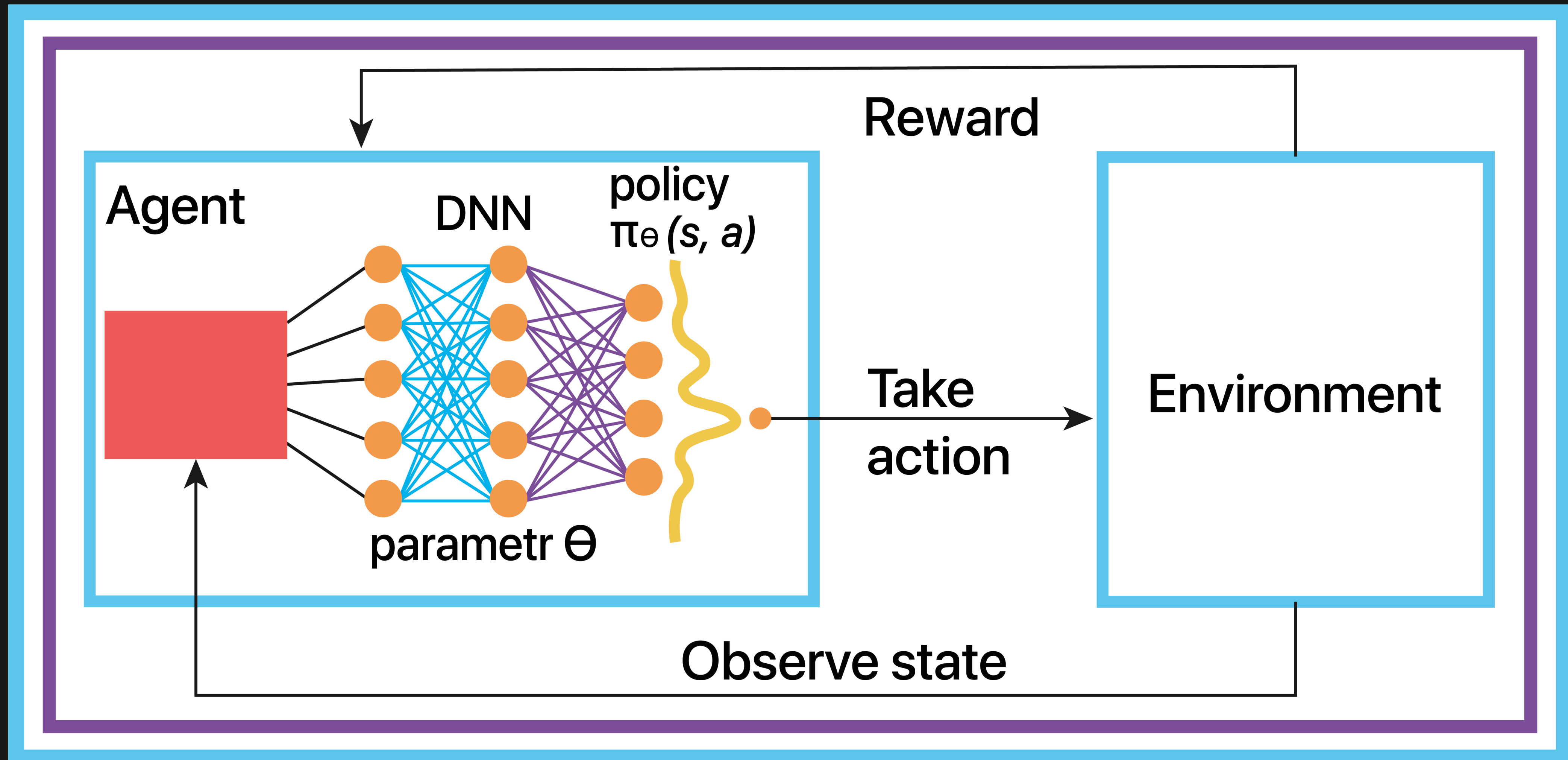
6. Кратность заправки доле от пустого места в баке

$$v_i = 0,25 q_i (s_{ub} - s_{i-1} + c_i), \quad \forall i \in I$$

7. Ограничение заправки в 100% от пустого места в баке

$$q_i \leq 4 b_i, \quad \forall i \in I$$

СХЕМА RL МОДЕЛИ



СРЕДА (ENVIRONMENT)

Из чего состоит Среда:

- > Набор маршрутов
- > Функция расчета вознаграждения (reward)
- > Состояние среды (observation)

Преобразование данных

- > Нормирование объемных показателей
- > Нормирование стоимости

Расчет вознаграждения

Затраты на заправку + штраф за нарушение минимального остатка + штраф за нарушение конечного остатка

Проблемы

- > Длина маршрута не постоянная
- > Порядок значений входных данных разнится

Потребление

Цены

[10, 15, 5, 20, 10, 15, 58, 61, 60, 57, 65, 650, 180, 350, 300, 200]

Максимальный, минимальный, текущий,
конечный остаток и минимальная
заправка



[0.015, 0.023, 0.008, 0.031, 0.015, 0, ... 0,
0.018, 0.07, 0.053, 0.0, 0.14, 0, ... 0,
1.0, 0.277, 0.538, 0.462, 0.308]

Потребление
Цены
Параметры

AGENT (DQN)

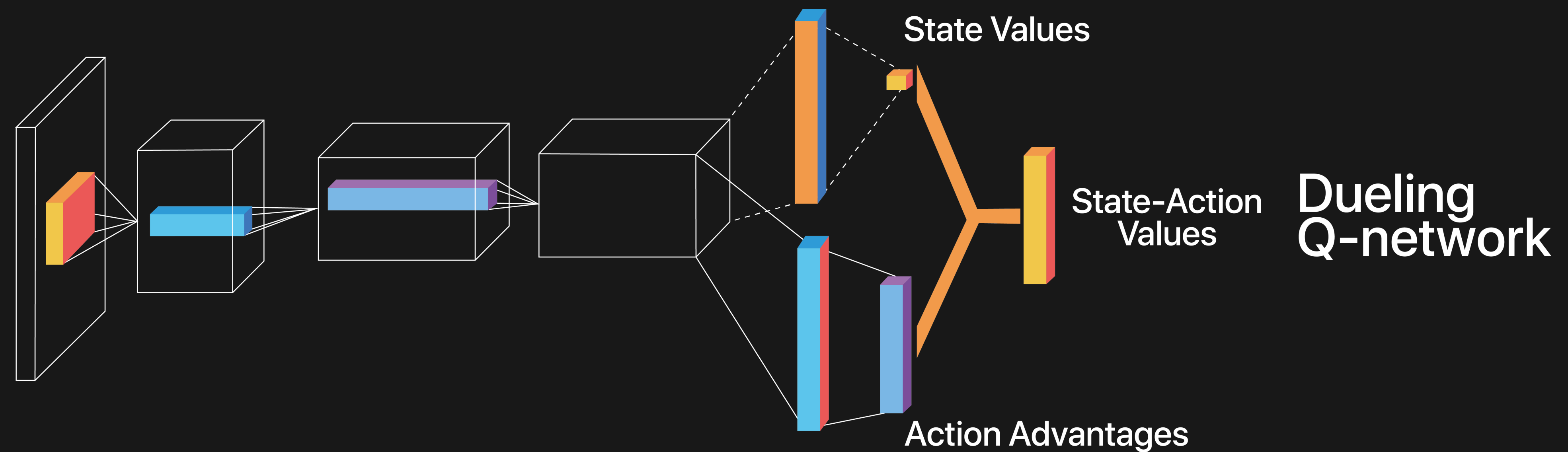
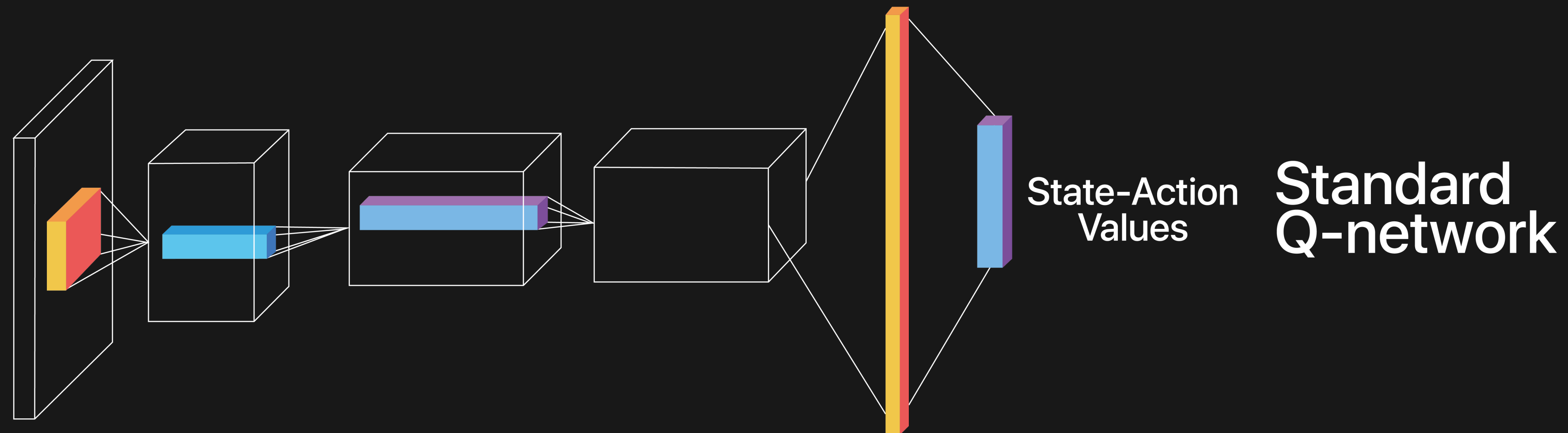
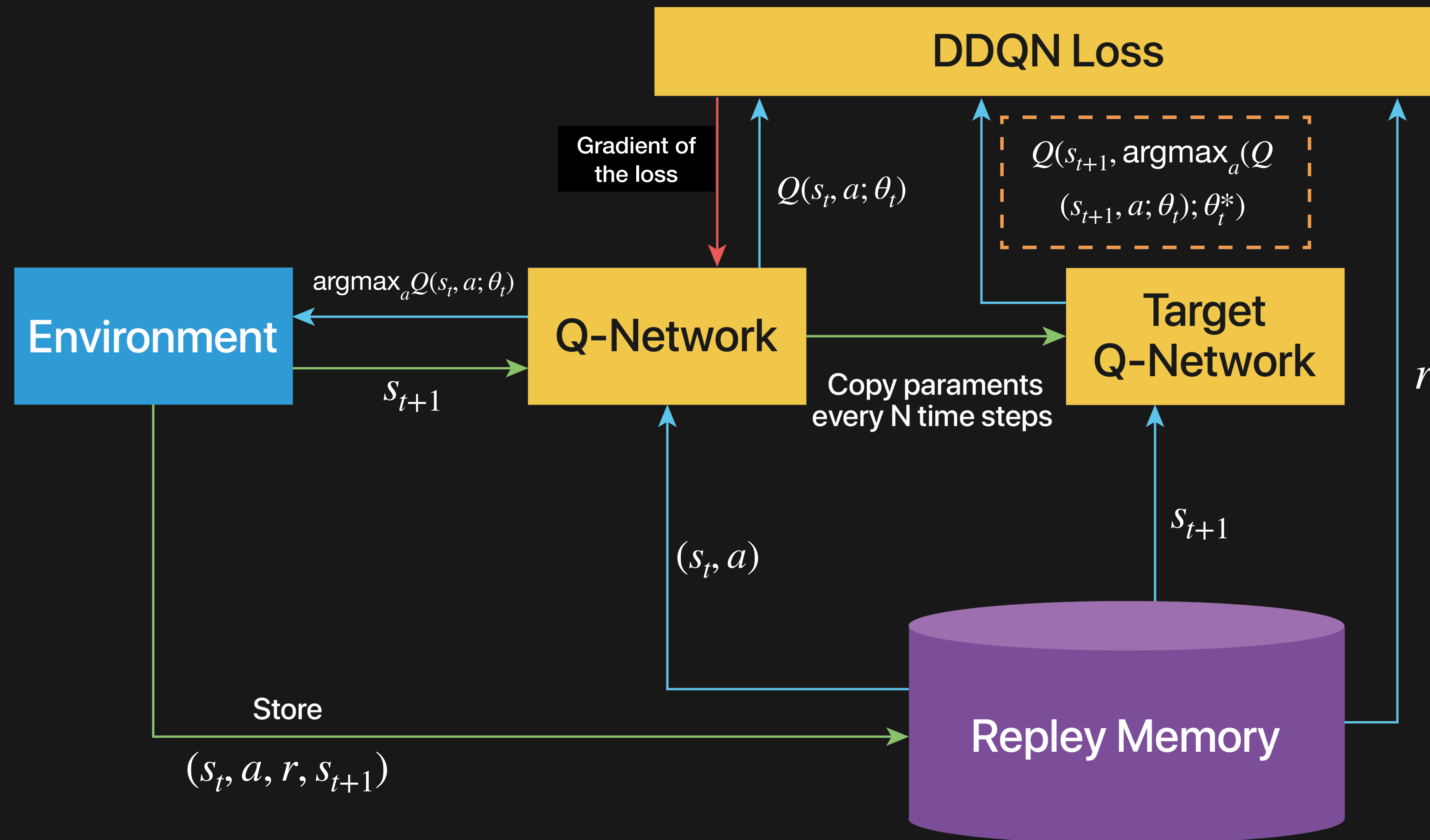


СХЕМА ОБУЧЕНИЯ DOUBLE DQN



ОБУЧЕНИЕ

LA Optimization

1. Инициализировать нейросеть $Q^*(s, a, \theta)$
2. Инициализировать таргет-сеть, положив $\theta_+ = \theta_-$.
3. Пронаблюдать s_0 из среды.
4. Для $k = 0, 1, 2, \dots$:
 1. с вероятностью ϵ выбрать действие a_k случайно, иначе жадно: $a_k = \operatorname{argmax}_{a_k} Q^*(s_k, a_k, \theta_-)$
 2. отправить действие a_k в среду, получить награду за шаг r_k и следующее состояние s_{k+1}
 3. добавить переход (s_k, a_k, r_k, s_{k+1}) в реплей буфер.
 4. если в реплей буфере скопилось достаточное число переходов, провести шаг обучения. Для этого сэмплируем мини-батч переходов (s, a, r, s') из буфера.
 5. для каждого перехода считаем целевую переменную: $y = r + \gamma \max_{a'} Q^*(s', a', \theta_-)$
 6. сделать шаг градиентного спуска для обновления θ_- , минимизируя $\sum (y - Q^*(s, a, \theta_-))^2$
 7. если k делится на 1000, обновить таргет-сеть: $\theta_+ = \theta_-$.

DQN

python

```
class LinearFFDQN(nn.Module):
    def __init__(self, shape, actions_n):
        super(LinearFFDQN, self).__init__()

        # Инициализация value-сети
        self.fc_val = nn.Sequential(
            nn.Linear(shape, 512), # входной слой
            nn.ReLU(),             # активация
            nn.Linear(512, 512),   # скрытый слой
            nn.ReLU(),             # активация
            nn.Linear(512, 1)      # выходной слой (одно значение)
        )

        # Инициализация advantage-сети
        self.fc_adv = nn.Sequential(
            nn.Linear(shape, 512), # входной слой
            nn.ReLU(),             # активация
            nn.Linear(512, 512),   # скрытый слой
            nn.ReLU(),             # активация
            nn.Linear(512, actions_n) # actions_n выходов
        )

    def forward(self, x):
        val = self.fc_val(x)      # вычисляем значение состояния
        adv = self.fc_adv(x)      # вычисляем преимущества действий
        # Возвращаем Q-значения по формуле:  $Q = V + (A - \text{mean}(A))$ 
        return val + (adv - adv.mean(dim=1, keepdim=True))
```

python

```
class DQNConv1D(nn.Module):
    def __init__(self, shape, actions_n):
        super(DQNConv1D, self).__init__()

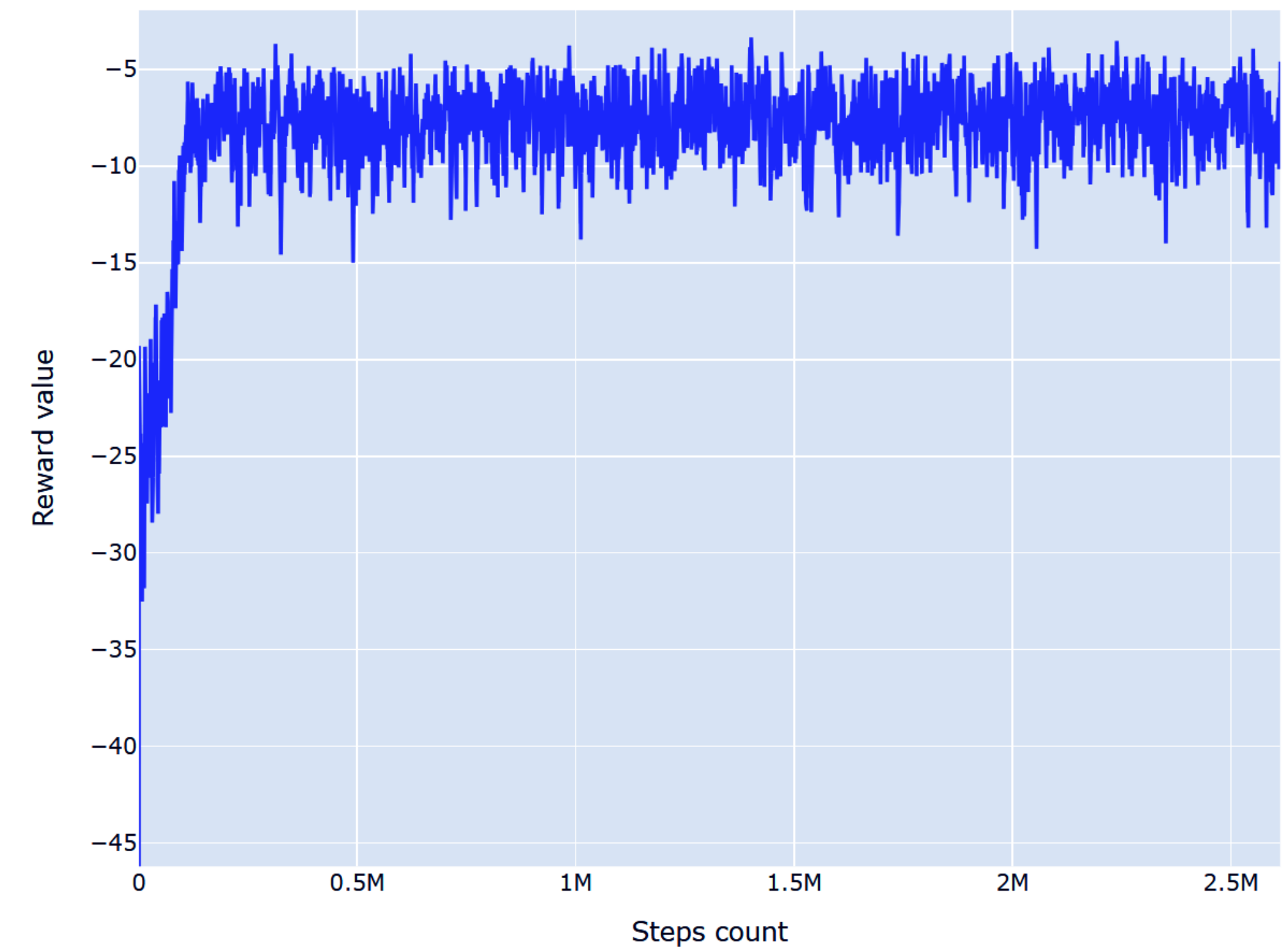
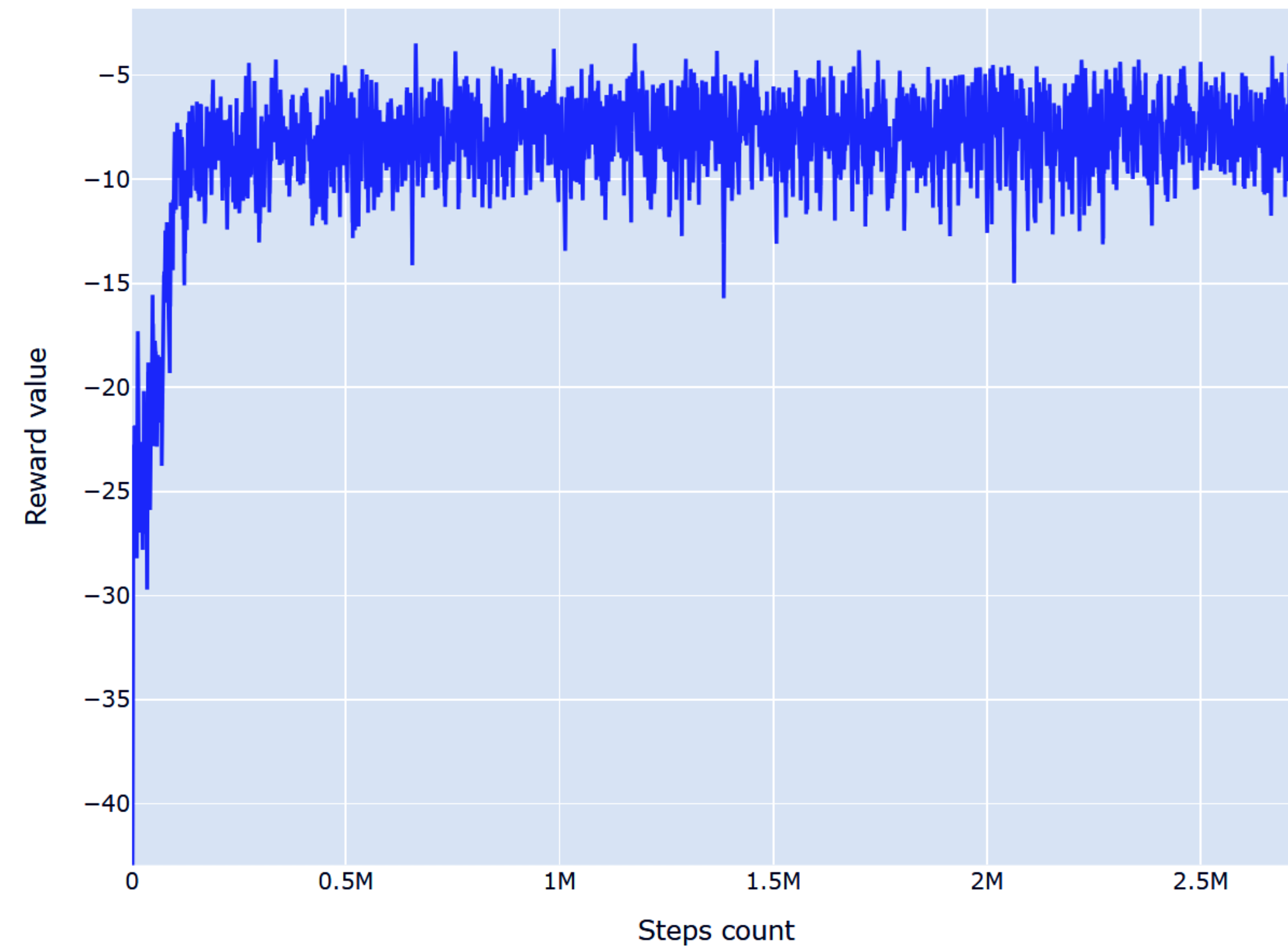
        # Сверточная часть сети
        self.conv = nn.Sequential(
            nn.Conv1d(shape[0], 128, 5), # 128 фильтров, размер ядра 5
            nn.ReLU(),
            nn.Conv1d(128, 128, 5),      # еще один сверточный слой
            nn.ReLU()
        )

        # Вычисление размера выхода сверточной части
        out_size = self._get_conv_out(shape)

        # Value-сеть
        self.fc_val = nn.Sequential(
            nn.Linear(out_size, 512),
            nn.ReLU(),
            nn.Linear(512, 1)
        )

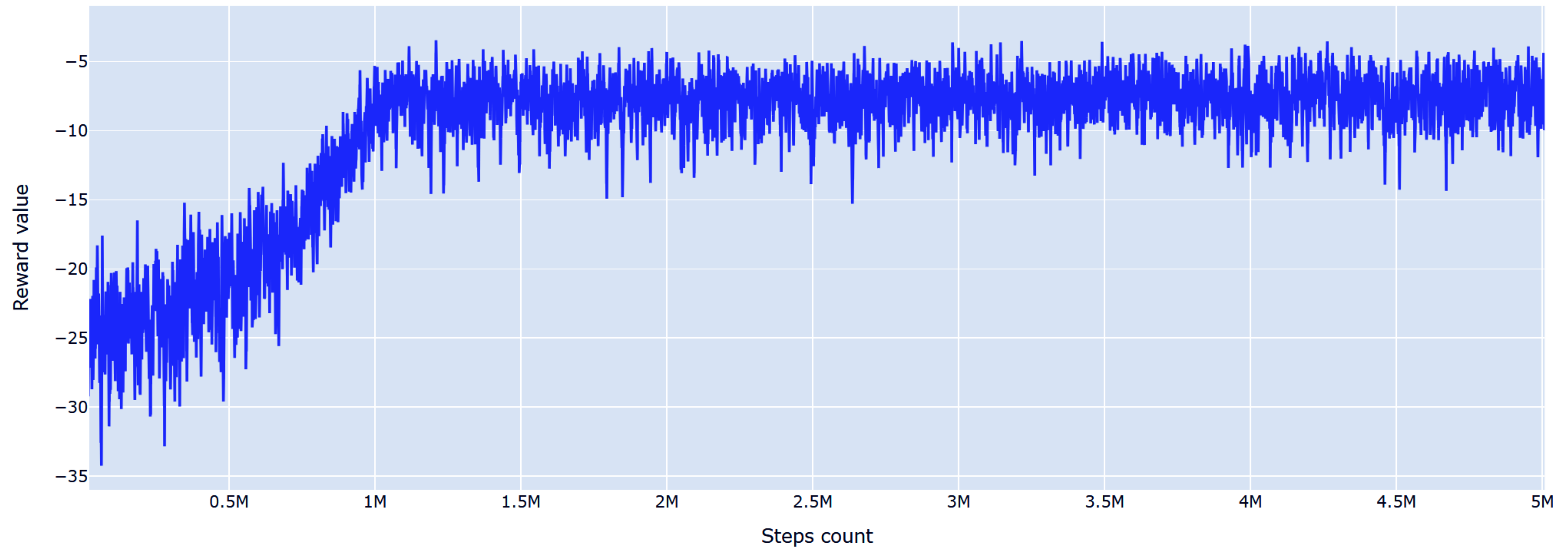
        # Advantage-сеть
        self.fc_adv = nn.Sequential(
            nn.Linear(out_size, 512),
            nn.ReLU(),
            nn.Linear(512, actions_n)
        )
```

ОБУЧЕНИЕ



ОБУЧЕНИЕ

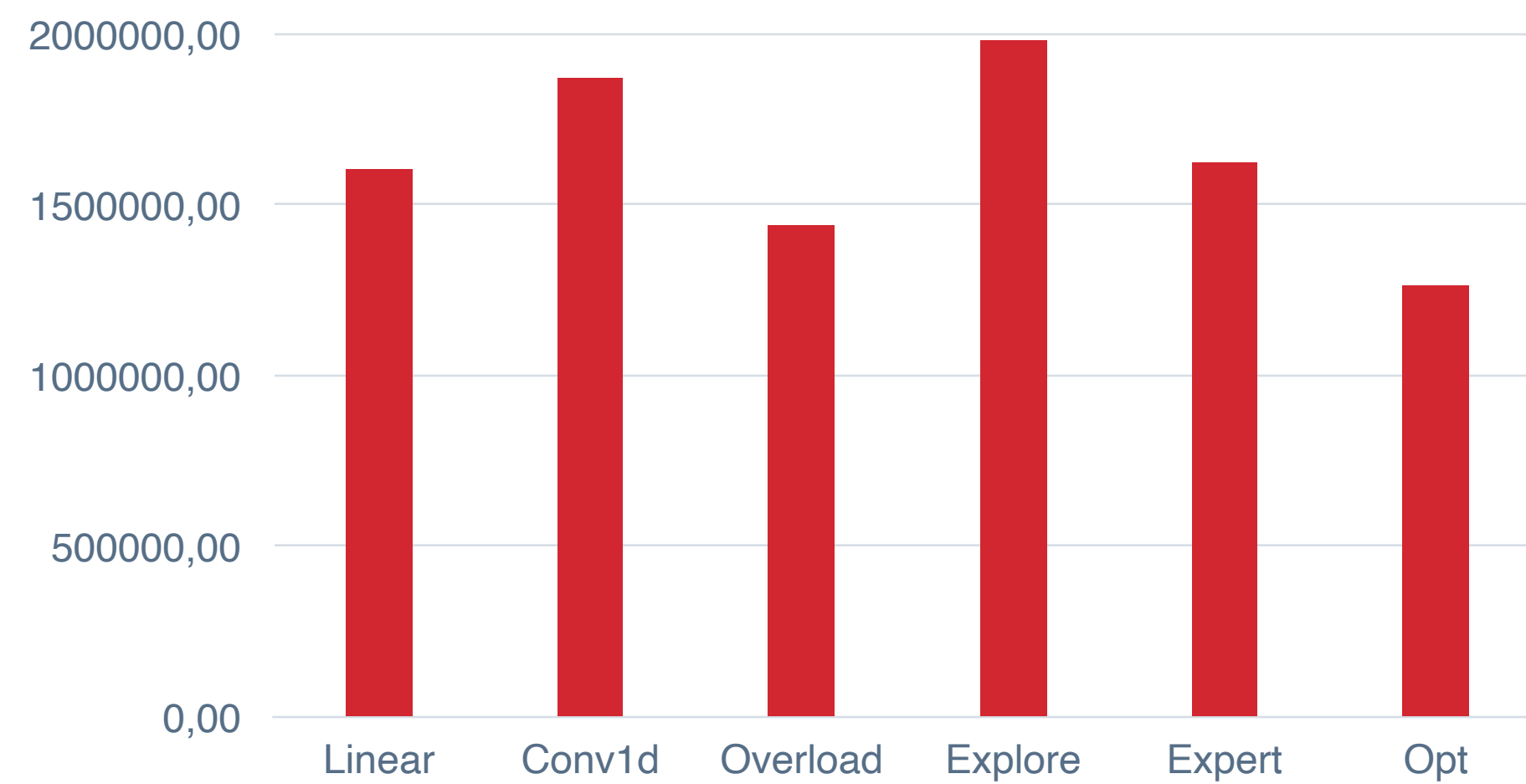
LA Optimization



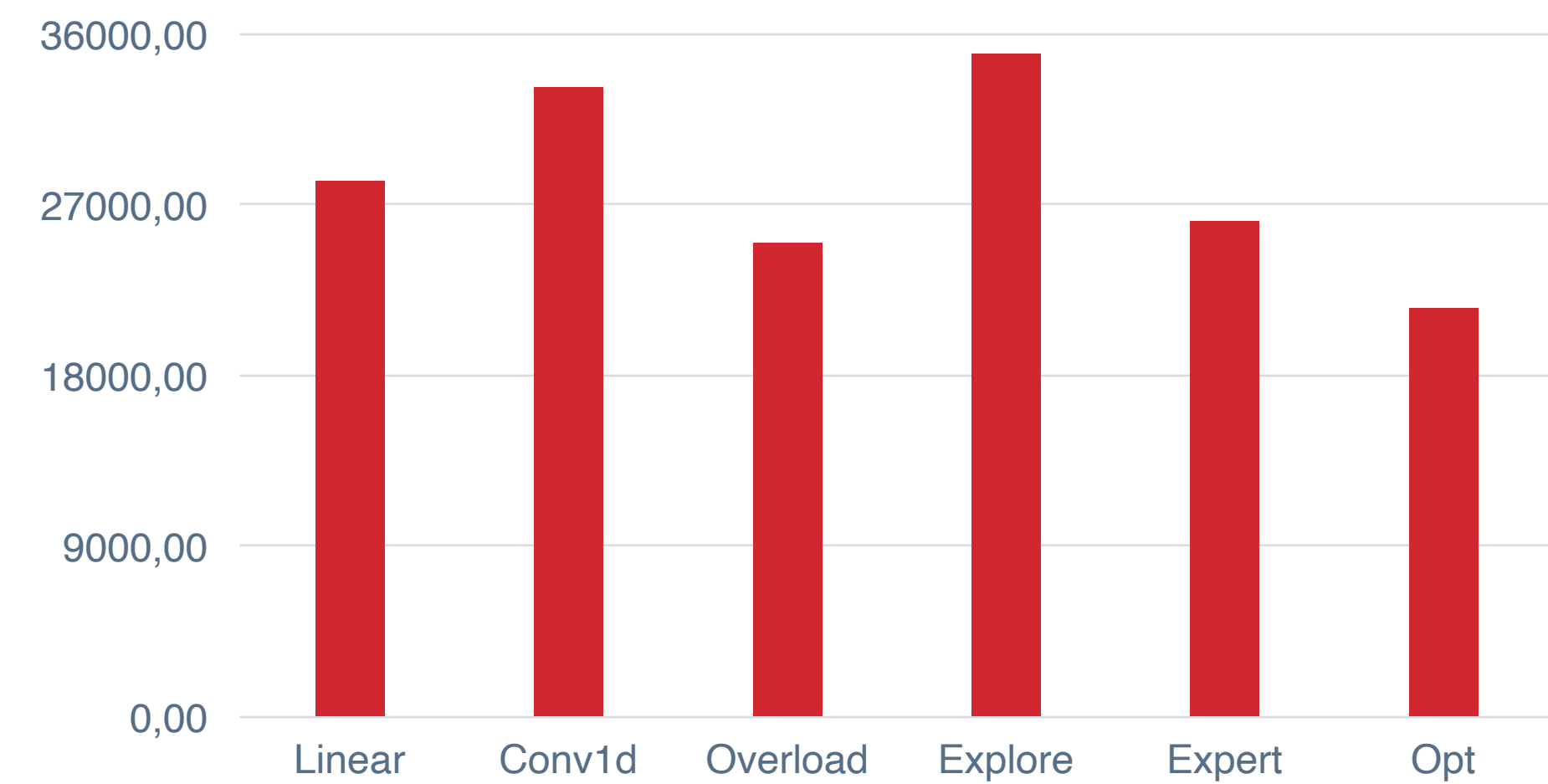
КАЧЕСТВО

LA Optimization

Общие затраты



Общий объем



КАЧЕСТВО

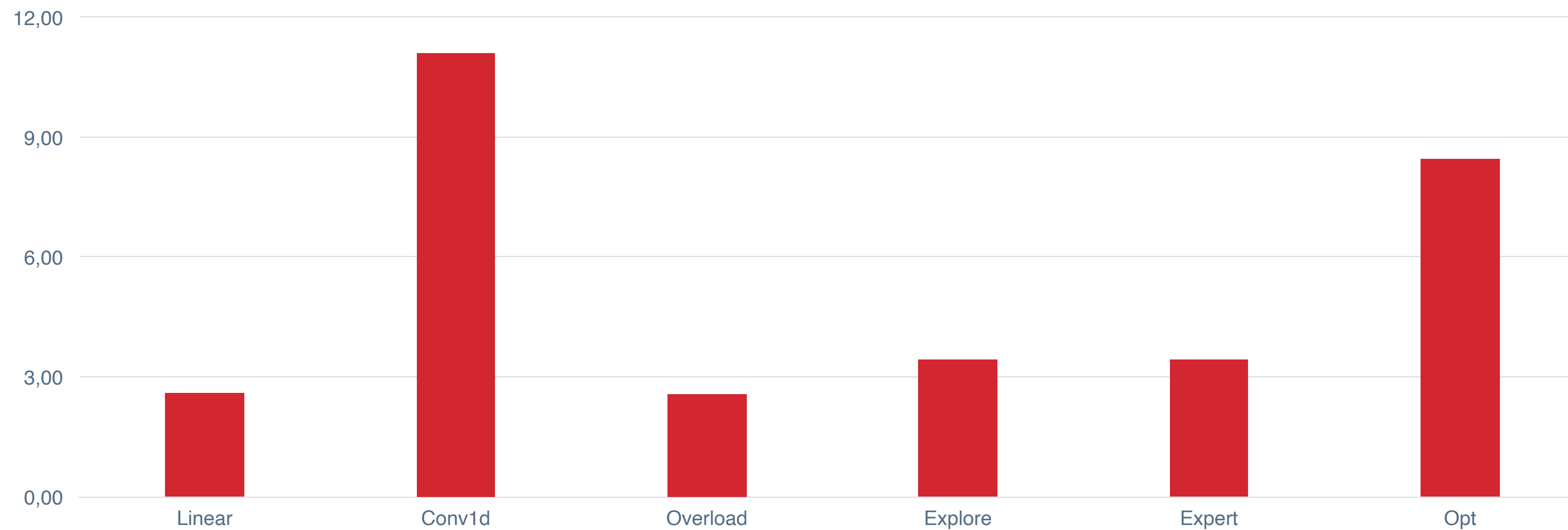
LA Optimization



КАЧЕСТВО

LA Optimization

Общая длительность расчета сек.



ЗАКЛЮЧЕНИЕ

Заключения

Нарушение минимального остатка:
фича или баг?

Общая стоимость или цена?

Резюме: RL плохо справился с
задачей

Много исследовательской работы

Исследования

Распределение награды после
завершения эпизода

Непрерывная функция действий

Усложнить DQN

Реальные данные для обучения

LA Optimization



СПАСИБО ЗА ВНИМАНИЕ

Ложкин Алексей
Консультант по задачам
исследованию операций

consult@lozkins.ru
+7 (952) 279-61-32

 @lozkins

 LA-optimization.ru